

Формат Индексного блока

[Прыжок на дно](#)Питер Диллинджер отредактировал эту страницу 8 июля 2025 г. · [10 правок](#)

Блок `index` содержит по одной записи на каждый блок данных, где ключ представляет собой строку `>=` — последний ключ в этом блоке данных и `<` — первый ключ в следующем блоке данных. Значение представляет собой `BlockHandle` (смещение и длину файла) для блока данных.

↪ Секционированный индекс

Если в качестве `IndexType` используется [kTwoLevelIndexSearch](#), то блок `index` представляет собой индекс второго уровня для индексных разделов, то есть каждая запись указывает на другой блок `index`, содержащий по одной записи на блок данных. В этом случае формат будет таким:

```
[index block - 1st level]
[index block - 1st level]
...
[index block - 1st level]
[index block - 2nd level]
```



↪ Ключ в индексных блоках

Как описано выше, ключ, хранящийся для блока, находится между последним ключом блока и первым ключом следующего блока. Обычно существует диапазон потенциальных ключей, удовлетворяющих этому условию. Выбор меньшего ключа может уменьшить размер индекса. Если `BlockBasedTableOptions.index_shortening` установлено в `kShortenSeparators` или `kShortenSeparatorsAndSuccessor`, последний ключ блока и первый ключ следующего блока будут переданы в `Comparator::FindShortestSeparator()` для поиска кратчайшего ключа-разделителя. Эта функция реализована во встроенных побитовых и обратных побитовых компараторах. Чтобы воспользоваться этой функцией, пользовательские компараторы должны ее реализовать.

Аналогичным образом индексный ключ для последнего блока определяется функцией `Comparator::FindShortSuccessor()`, которая возвращает любой ключ, который больше или равен последнему ключу последнего блока. Пользователям также необходимо реализовать эту функцию для пользовательского компаратора, чтобы воспользоваться функцией экономии памяти.

Значение по умолчанию для этой опции — `kShortenSeparators`, что сокращает индексный ключ для всех блоков, кроме последнего. Это связано с тем, что последний ключ оказывает минимальное влияние на размер индекса, но может положительно сказаться на предотвращении чтения последних блоков данных.

Значение `kNoShortening` также можно использовать вместе с `index_type = kBinarySearchWithFirstKey`, чтобы предотвратить чтение некоторых блоков с помощью специальной функции, о которой мы расскажем ниже.

🔗 Индивидуальный Индексный блок

До версии 5.14 RocksDB, `BlockBasedTableOptions::format_version = 2`, формат блоков индекса и данных был одинаковым. В блоках индекса использовался тот же формат ключа `< user_key , seq >`, но со специальными значениями `< offset , size >`, которые указывали на блоки данных. В отличие от блоков данных, параметр, определяющий размер блока перезапуска, — это

`BlockBasedTableOptions.index_block_restart_interval`, а не `BlockBasedTableOptions.block_restart_interval`. Значение по умолчанию — 1, а не 16, как для блоков данных. Таким образом, значение по умолчанию требует относительно больших затрат памяти. Установка значения 8 или 16 обычно позволяет уменьшить размер индексного блока вдвое, но в зависимости от рабочей нагрузки может возрасти нагрузка на центральный процессор. `format_version = 3, 4` — еще один оптимизированный размер, но несовместимый с будущими версиями формат индексных блоков.

- `format_version = 3` (начиная с версии RocksDB 5.15): в большинстве случаев порядковый номер `seq` не нужен для ключей в индексных блоках. В таких случаях этот `format_version` параметр пропускает кодирование порядкового номера и устанавливает `index_key_is_user_key` в `TableProperties`, что позволяет читателю понять, как декодировать индексный блок.
- `format_version = 4` (начиная с версии RocksDB 5.16): изменяет формат индексных блоков, используя дельта-кодирование значений индекса, которые являются дескрипторами блоков. Это позволяет экономить на кодировании `BlockHandle::offset` неглавных индексных записей в каждом интервале перезапуска. Если используется, то задается `TableProperties::index_value_is_delta_encoded` — параметр, который используется считывающим устройством для определения способа декодирования индексного блока. Формат каждого ключа: `(shared_size,`

non_shared_size, shared, non_shared). Формат каждого значения, то есть дескриптора блока, — {Varint64(offset), Varint64(size)} при значении shared_size, равном 0, включая первую запись в каждой точке перезапуска. В противном случае используется формат Varsignedint64(delta-sz), где delta-sz = размер текущего дескриптора блока — размер предыдущего дескриптора блока.

Формат индекса в format_version=4 будет следующим:

```
restart_point 0: k, v (off, sz), k, v (delta-sz), ..., k, v (delta-sz)
restart_point 1: k, v (off, sz), k, v (delta-sz), ..., k, v (delta-sz)
...
restart_point n-1: k, v (off, sz), k, v (delta-sz), ..., k, v (delta-sz)
where, k is key, v is value, and its encoding is in parenthesis.
```



Формат применяется в случае, когда index_type != kBinarySearchWithFirstKey .
Случай kBinarySearchWithFirstKey описан в следующем разделе.

↪ index_type == kBinarySearchWithFirstKey

Особенность index_type == kBinarySearchWithFirstKey заключается в том, что RocksDB может видеть первый ключ блока данных, не считывая его с диска. Благодаря этой функции RocksDB знает первый ключ блока и может отложить его чтение до тех пор, пока значение не понадобится. Это позволяет эффективно предотвращать операции ввода-вывода и другие накладные расходы при выполнении некоторых особых задач. Например, при сканировании диапазона, когда RocksDB объединяет пары «ключ-значение» из нескольких источников с помощью структуры кучи, чтение блока данных из SST-файла может быть отложено до тех пор, пока блок не достигнет вершины кучи (а может и вовсе не потребоваться, если он так и не достигнет вершины).

Если используется этот параметр, то для каждой записи в индексном блоке после части BlockHandle (смещение, размер) сохраняется первый ключ блока в виде строки с префиксом длины. Например, version_format = 5 будет иметь следующий формат:

```
restart_point 0:
  k (block_key), v (block_offset, block_size, size_of_first_key,
first_key)
  k (block_key), v (delta_size, size_of_first_key, first_key)
  k (block_key), v (delta_size, size_of_first_key, first_key)
restart_point 1:
  k (block_key), v (block_offset, block_size, size_of_first_key,
first_key)
  k (block_key), v (delta_size, size_of_first_key, first_key)
```



```
k (block_key), v (delta_size, size_of_first_key, first_key)
...
```

It's similar to other format versions and restart block size.

► **Pages** 167

Contents

- [RocksDB Wiki](#)
- [Overview](#)
- [RocksDB FAQ](#)
- [Terminology](#)
- [Requirements](#)
- [Contributors' Guide](#)
- [Release Methodology](#)
- [RocksDB Users and Use Cases](#)
- [RocksDB Public Communication and Information Channels](#)
- [Basic Operations](#)
 - [Iterator](#)
 - [Prefix seek](#)
 - [SeekForPrev](#)
 - [Tailing Iterator](#)
 - [Compaction Filter](#)
 - [Multi Column Family Iterator](#)
 - [Read-Modify-Write \(Merge\) Operator](#)
 - [Column Families](#)
 - [Creating and Ingesting SST files](#)
 - [Single Delete](#)
 - [SST Partitioner](#)
 - [Low Priority Write](#)
 - [Time to Live \(TTL\) Support](#)
 - [Transactions](#)
 - [Snapshot](#)
 - [DeleteRange](#)
 - [Atomic flush](#)
 - [Read-only and Secondary instances](#)
 - [Approximate Size](#)
 - [User-defined Timestamp](#)
 - [Wide Columns](#)
 - [BlobDB](#)
 - [Online Verification](#)
- **Options**
 - [Setup Options and Basic Tuning](#)
 - [Option String and Option Map](#)
 - [RocksDB Options File](#)

- [MemTable](#)
- [Journal](#)
 - [Write Ahead Log \(WAL\)](#)
 - [Write Ahead Log File Format](#)
 - [WAL Recovery Modes](#)
 - [WAL Performance](#)
 - [WAL Compression](#)
 - [MANIFEST](#)
 - [Track WAL in MANIFEST](#)
- **Cache**
 - [Block Cache](#)
 - [SecondaryCache \(Experimental\)](#)
- [Write Buffer Manager](#)
- [Compaction](#)
 - [Leveled Compaction](#)
 - [Universal compaction style](#)
 - [FIFO compaction style](#)
 - [Manual Compaction](#)
 - [Subcompaction](#)
 - [Choose Level Compaction Files](#)
 - [Managing Disk Space Utilization](#)
 - [Trivial Move Compaction](#)
 - [Remote Compaction](#)
- **SST File Formats**
 - [Block-based Table Format](#)
 - [PlainTable Format](#)
 - [CuckooTable Format](#)
 - [External Table](#)
 - [Index Block Format](#)
 - [Bloom Filter](#)
 - [Data Block Hash Index](#)
- [IO](#)
 - [Rate Limiter](#)
 - [SST File Manager](#)
 - [Direct I/O](#)
- [Compression](#)
 - [Dictionary Compression](#)
- [Full File Checksum and Checksum Handoff](#)
- [Background Error Handling](#)
- [Huge Page TLB Support](#)
- [Tiered Storage \(Experimental\)](#)
- **Logging and Monitoring**
 - [Logger](#)
 - [Statistics](#)
 - [Compaction Stats and DB Status](#)
 - [Perf Context and IO Stats Context](#)
 - [EventListener](#)
- [Known Issues](#)
- [Troubleshooting Guide](#)
- [Tests](#)
 - [Stress Test](#)

- [Fuzzing](#)
- [Benchmarking](#)
- **Tools / Utilities**
 - [Administration and Data Access Tool](#)
 - [How to Backup RocksDB?](#)
 - [Replication Helpers](#)
 - [Checkpoints](#)
 - [How to persist in-memory RocksDB database](#)
 - [Third-party language bindings](#)
 - [RocksDB Trace, Replay, Analyzer, and Workload Generation](#)
 - [Block cache analysis and simulation tools](#)
 - [IO Tracer and Parser](#)
- **Implementation Details**
 - [Delete Stale Files](#)
 - [Partitioned Index/Filters](#)
 - [WritePrepared-Transactions](#)
 - [WriteUnprepared-Transactions](#)
 - [How we keep track of live SST files](#)
 - [How we index SST](#)
 - [Merge Operator Implementation](#)
 - [RocksDB Repairer](#)
 - [Write Batch With Index](#)
 - [Two Phase Commit](#)
 - [Iterator's Implementation](#)
 - [Simulation Cache](#)
 - [\[To Be Deprecated\] Persistent Read Cache](#)
 - [DeleteRange Implementation](#)
 - [unordered_write](#)
- **Extending RocksDB**
 - [RocksDB Configurable Objects](#)
 - [The Customizable Class](#)
 - [Object Registry](#)
- **RocksJava**
 - [RocksJava Basics](#)
 - [Logging in RocksJava](#)
 - [JNI Debugging](#)
 - [RocksJava API TODO](#)
 - [RocksJava Performance on Flash Storage](#)
 - [Tuning RocksDB from Java](#)
- **Performance**
 - [Performance Benchmarks](#)
 - [In Memory Workload Performance](#)
 - [Read-Modify-Write \(Merge\) Performance](#)
 - [Delete A Range Of Keys](#)
 - [Write Stalls](#)
 - [Pipelined Write](#)
 - [MultiGet Performance](#)
 - [Tuning Guide](#)
 - [Memory usage in RocksDB](#)
 - [Speed-Up DB Open](#)

- [Implement Queue Service Using RocksDB](#)
- [Asynchronous IO](#)
- [Off-peak in RocksDB](#)
- [Projects Being Developed](#)
- **Misc**
 - [Building on Windows](#)
 - [Developing with an IDE](#)
 - [Open Projects](#)
 - [Talks](#)
 - [Publication](#)
 - [Features Not in LevelDB](#)
 - [How to ask a performance-related question?](#)
 - [Articles about Rocks](#)

Clone this wiki locally

`https://github.com/facebook/rocksdb.wiki.git`

